In-class exercise: code smells

In-class exercise — problems

Each slide shows a short Python snippet.

In-class exercise — problems

Each slide shows a short Python snippet.

For each one:

In-class exercise — problems

Each slide shows a short Python snippet.

For each one:

1. Name the smell (or smells).

In-class exercise — problems

Each slide shows a short Python snippet.

For each one:

1. Name the smell (or smells).
2. Sketch a refactored version.

In-class exercise — problems

Each slide shows a short Python snippet.

For each one:

1. Name the smell (or smells).
2. Sketch a refactored version.

In-class exercise — problems

Each slide shows a short Python snippet.

For each one:

1. Name the smell (or smells).
2. Sketch a refactored version.

Work in pairs. ***

Exercise 1

```
def f(x, s, n, k, a, b, flag):
    from math import sqrt, pi, exp, comb
    if flag == "normal":
        return (1/(sqrt(2*pi)*s)) * exp(-0.5*((x-a)/s)**2)
    elif flag == "binomial":
        return comb(n, k) * (x**k) * ((1-x)**(n-k))
    elif flag == "beta":
        from math import gamma
        return (x**(a-1)*(1-x)**(b-1))/(gamma(a)*gamma(b)/gamma(a+b))
```

Exercise 2

```
def run(data, conds, sbjs, dprimes, criteria, mdl, fit, r, p, out):
    r = []
    for s in sbjs:
        sd = [data[i] for i in range(len(data)) if conds[i] == s]
        dp = sum([x[0] for x in sd]) / len(sd)
        cr = sum([x[1] for x in sd]) / len(sd)
        dprimes.append(dp)
        criteria.append(cr)
    fit = mdl(dprimes, criteria)
    out = "significant" if fit.pvalue < 0.05 else "not significant"
    return out
```

Exercise 3

```
def c(h, f):
    from scipy.stats import norm
    return norm.ppf(h) - norm.ppf(f)

def b(h, f):
    from scipy.stats import norm
    return -0.5 * (norm.ppf(h) + norm.ppf(f))
```

```
def compute_sensitivity_index_for_signal_detection_analysis(hr, far):
    from scipy.stats import norm
    # Compute d-prime using the standard SDT formula
    # hr is hit rate, far is false alarm rate
    z_hit = norm.ppf(hr)   # z-score for hit rate
    z_far = norm.ppf(far)  # z-score for false alarm rate
    dp = z_hit - z_far     # d-prime is the difference
    return dp              # return the result
```

Exercise 5

```python
import numpy as np

def analyze(results):
    results = [r for r in results if r is not None]
    results = [r * 1000 for r in results]
    results = np.array(results)
    results = results[results < np.percentile(results, 95)]
    results = results - results.mean()
    return results
```

Exercise 6

```
def run_experiment(participant_id, session_number,
                   condition_label, stimulus_list,
                   response_key_map, timeout_ms,
                   practice_trials, fixation_duration_ms,
                   feedback_enabled, log_file_path):
    ...
```

Exercise 7

```
hits_a  = sum(1 for t in block_a if t.correct)
total_a = len(block_a)
hr_a    = hits_a / total_a

hits_b  = sum(1 for t in block_b if t.correct)
total_b = len(block_b)
hr_b    = hits_b / total_b
```

Exercise 8

```
def integrand_uniform(self, p):
    return self.likelihood(p) * self.uniform_prior(p)

def integrand_beta(self, p):
    return self.likelihood(p) * self.beta_prior(p)

def integrand_jeffreys(self, p):
    return self.likelihood(p) * self.jeffreys_prior(p)
```

Exercise 9

```
def compute_median(trial_list):
    trial_list.sort()
    n = len(trial_list)
    return trial_list[n // 2]

rts = [0.42, 0.31, 0.58, 0.29]
m   = compute_median(rts)
print(rts)  # [0.29, 0.31, 0.42, 0.58] — caller's list was mutated
```

Exercise 10

```
class SignalDetection:

    # Set the d-prime value
    def set_dprime(self, value):
        self.dprime = value  # store d-prime

    # Get the d-prime value
    def get_dprime(self):
        return self.dprime  # return the threshold
```

Exercise 11

```
summary = {
    s: sum(t.rt for t in trials if t.subject==s)
       / len([t for t in trials if t.subject==s])
    for s in subjects
}
```

Exercise 12

```
from scipy.stats import norm

def analyse(sid, hit, far, mrt, srt, age, grp):
    if not (0 <= hit <= 1):
        raise ValueError("hit out of range")
    dp = norm.ppf(hit) - norm.ppf(far)
    return sid, dp, mrt / srt
```